

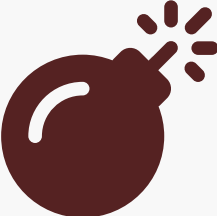
# System Architecture: Business Case Priority Intake Form

This document provides a technical overview of the system architecture for the **Business Case Priority Intake Form** application.

## 1. System Topology and Tier Architecture

The application is structured as a three-tier decoupled architecture:

- 1. **User Interface / Presentation Tier:** Streamlit web application.
- 2. **Agent / Processing Tier:** Google ADK 2.0 Graph Workflow.
- 3. **Storage / Data Warehouse Tier:** Databricks SQL Warehouse (Delta Lake).

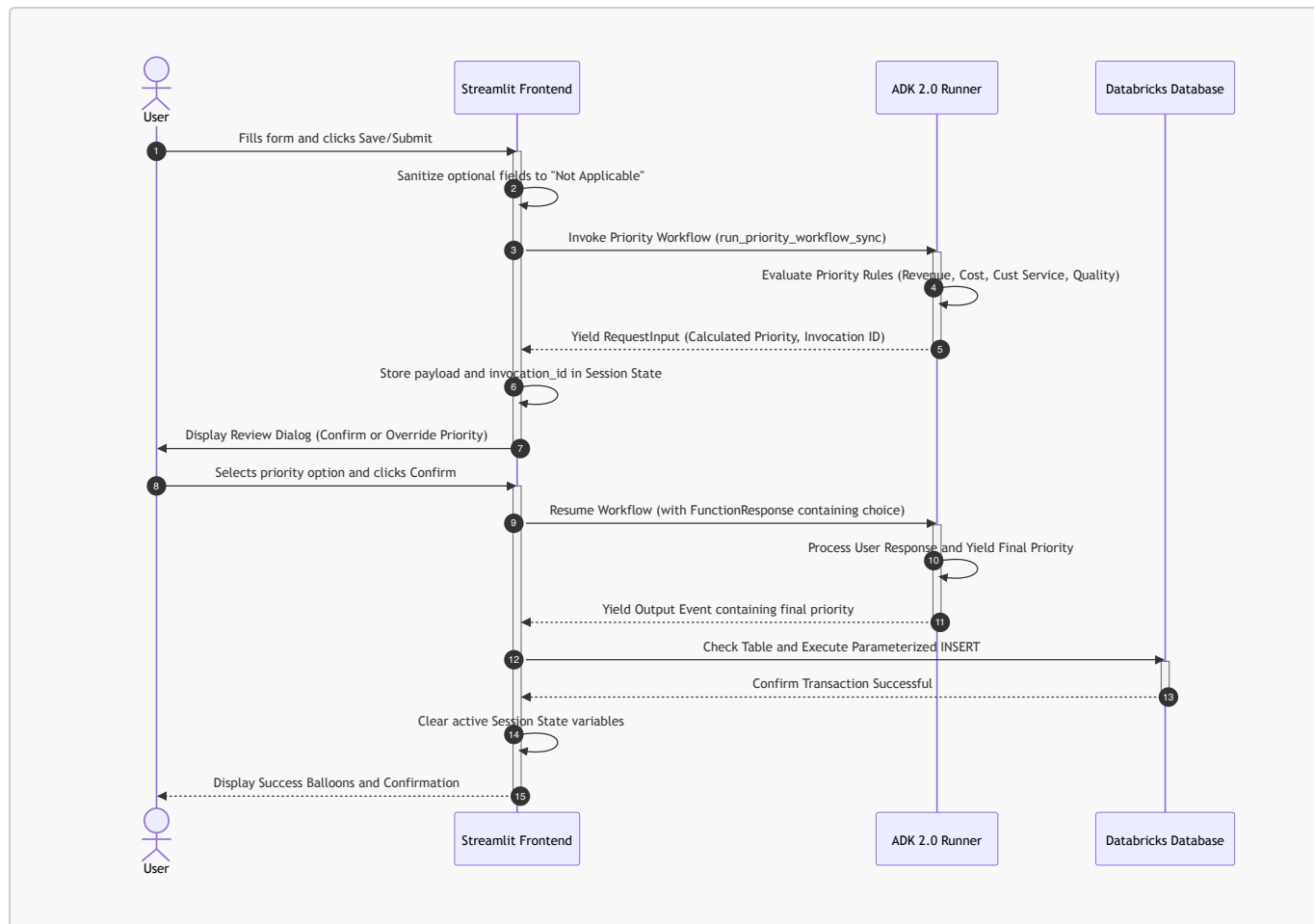


### Syntax error in text

mermaid version 11.16.0

## 2. Dynamic Integration and Interaction Flow

The interaction logic is designed to support asynchronous human-in-the-loop validation within a synchronous frontend execution flow.



### 3. Tier Component Specifications

#### 3.1. Presentation Tier (Streamlit)

- **Design and Theme:** Overridden globally via `.streamlit/config.toml` to enforce a clean light mode layout with white backgrounds (`#ffffff`), slate text (`#0f172a`), and indigo UI highlights (`#4f46e5`).
- **Session State Management:** Tracks active state across user interactions:
  - Form inputs (e.g. project name, problem solving).
  - Dynamic list of KPI objects.
  - Integration configuration parameters (Databricks settings, Gemini API keys).
  - Active Priority Workflow run session parameters (`priority_run_session_id`, `pending_priority_confirmation` containing the active `invocation_id` and the database payload).
- **Validation and Progress:** Computes input completion status dynamically. Only `project_name` and `problem_solving` (Project Description) are required. All other text fields default to "Not Applicable" if empty.

#### 3.2. Orchestration Tier (Google ADK 2.0)

- **Agent Scaffolding:** Utilizes the `Workflow` class from `google.adk.workflow`.
- **Graph Definition:** Configured with a `START` node connected to a single processing node:
  - Processing Node: `determine_and_confirm_priority` decorated with `@node(rerun_on_resume=True)`.

- Schemas: Input is validated via a Pydantic `ProjectData` model.
- **Interrupt State Handling:** Uses the `RequestInput` class from `google.adk.events.request_input`.
  - If the input is not yet verified, it yields `RequestInput` with the computed value, interrupting workflow execution.
  - Resumption is executed via `runner.run_async()` by passing the active `invocation_id` and a `FunctionResponse` matching the interrupt signature.

### 3.3. Database and Storage Tier (Databricks)

- **Connection Interface:** `databricks-sql-connector` library executing over Databricks SQL Warehouse instances.
- **Authentication:** Supports both token-based login and automated OAuth credential resolution via the Databricks SDK (`databricks-sdk`) when deployed inside Databricks Apps.
- **Schema Evolution and Safety:**
  - Employs `CREATE DATABASE IF NOT EXISTS` and `CREATE TABLE IF NOT EXISTS` commands to ensure target storage paths are validated on run.
  - Executes an `ALTER TABLE ... ADD COLUMN IF NOT EXISTS priority STRING` migration query during connection validation to support schema updates without breaking existing data.
  - All database operations utilize parameterized queries (`?` syntax placeholders) to protect against SQL injection vulnerabilities.